EEE4120F

High Performance Embedded Systems

PRACTICAL 3

Shared and Distributed Memory Parallel Processing

Course Team

Course Convener: Associate Professor Simon Winberg

Teaching Assistant: Travimadox Webb

Practical 3

Shared and Distributed Memory Parallel Processing

Summary of Deliverables

- **Duration:** 2 Weeks (17th Mar - 7th April)
- **Report:** A short technical report in IEEE Confrence Format (PDF, Max 4 Pages) via Gradescope assignment **Practical 3 Report**.
- **Code:** Submit your code files(wariara_freights_route.c from MPI folder and wariara_freights_route.c from OpenMp folder) to Gradescope assignment **Practical 3 Oral Exam**.

Note on Environments

- **Linux:** You're mostly fine.
- **Windows:** Use **WSL**. It's much easier than trying to convince Windows to act like a real dev environment. Although you are welcome to try(For people who love to be in toxic relationships). I advise that energy be spent on better things like actually completing this prac or building your MARIS project for Stephen.
- **macOS:** You advised to **brew install gcc**. Why? Because Apple decided that their **gcc** command should actually just be an alias for **Clang**. It's basically gaslighting in compiler form. There are two options to proceed , install the real gcc or stick to clang but remember you'll need to install one key library(for openmp) before it works. Definitely am not telling you, ask ChatGPT LOL

1 Introduction

In this practical, you will investigate the optimisation of parallel programs in both **shared-memory** and **distributed-memory** computing environments. The practical focuses on implementing and analysing parallel solutions in **C** using **OpenMP** for shared-memory parallelism and **MPI** for distributed-memory parallelism, and comparing their performance characteristics.

In a previous practical (Prac 2), MATLAB's built-in parallel constructs abstracted much of the complexity of parallel execution. In contrast, this practical requires you to explicitly manage parallelism, memory access, and communication. This provides deeper insight into how parallel programs are designed, implemented, and optimised at a lower level.

Core Concepts

1. **Distributed-Memory Parallel Computing:** Each processing element has its own private memory space, and processes communicate explicitly via message passing (MPI).
2. **Shared Memory Parallel Computing:** Multiple threads operate within a common address space, coordinating access to shared data using OpenMP constructs.
3. **Combinatorial Optimisation Problems**: Problems in which the objective is to find an optimal solution from a finite but typically very large set of possible combinations.

2 Problem

Assume you are working as a Systems Engineer for Wariara Freights Inc. Recently, the company has decided to switch to Autonomous Electronic Trucks (AETs). These AETs operate as follows:

1. They operate on a single charge logic to minimise downtime.
2. AET is loaded at the Central Distribution Hub (City 1) and assigned a manifest of N regional client drop-off locations that it must visit.
3. AET must visit every client drop-off location exactly once to drop off their specific cargo.
4. AET does not return to the Central Distribution Hub (City 1) after the last delivery. Instead, it parks at the final drop-off location to recharge overnight.
5. Energy efficiency is critical for the operation of AETs; thus, the minimum total energy consumption (in kWh) for the entire trip is prioritised.
6. There is only one client per client location. (Very simplistic but, this makes the practical doable in the allocated time)
7. The road network is fully connected, i.e the truck can travel directly from any client to another client. The challenge is that the energy costs between client locations vary significantly due to distance, altitude and traffic patterns. As the number of clients, N, increases, the number of possible delivery routes grows factorially (N!).

Your Chief Engineer instructs you to write a program to calculate the delivery sequence that minimises total energy consumption.

The input to the problem is provided as a matrix. An element of the matrix, E[i][j], gives the energy cost between client locations i (City i) and j (City j). Thus, the input to your program is a file organised as follows, with N as the number of client locations:

Input

$$\begin{array}{llll} N & & & \\ E[1][2] & & & \\ E[1][3] & E[2][3] & & \\ E[1][4] & E[2][4] & E[3][4] & \\ \vdots & \vdots & \vdots & \ddots \\ E[1][N] & E[2][N] & E[3][N] & \dots \end{array}$$

An example where $N = 4$ is shown below:

Input

$$\begin{array}{lll} 4 & & \\ 54 & & \\ 76 & 30 & \\ 24 & 51 & 64 \end{array}$$

The program's output should be an ordered list of client locations (numbers between 1 and N). As you might have realised, there are two equivalent permutations; the total energy consumption will be identical whether the path is travelled forward or backward. For example, if

the optimal sequence is 1,3,2,4, the reverse 4,2,3,1 will still give the same energy consumption. Either one is acceptable as the final asner for the most optimal route.

Due to time and prototyping constraints, your Chief Engineer has restricted you to using the **Branch-and-Bound** optimisation algorithm to find the optimal route. Because this calculation must happen rapidly before the truck departs, you are required to parallelise the solver to maximise performance.

For more information on the Branch and Bound Algorithm check the Appendix

3 Tasks

3.1 Task 1: Shared Memory Parallel Programming using OpenMP

Tip

You might want to refer to the practical tutorial, specifically the OpenMP section, before starting.

Imagine the routing software runs locally on the AET's Nvidia Jetson Orin. Your task is to implement the solver using OpenMP. You will use your PC to emulate the AET's onboard processing environment.

You are required to do the following:

1. Implement a parallel Branch-and-Bound program for the minimum-energy-consumption problem, aiming to achieve the best possible speed-up. Your program should accept the command-line arguments -p and -i to specify the number of processors used and the input file, respectively. It should also use the input file format described above. (File loading and command-line parsing are already implemented for you in the starter code).
2. Evaluate the performance of your program by measuring the following times using the provided gettime() functions that return the current time in seconds
   - Initialisation Time(T_{init}): The time required to read all the command-line arguments and create the separate processes.
   - Computation time(T_{comp}): Measures strictly the time to compute the answer, i.e you start timing when the main computation starts and end when the answer has been calculated.

Tip

$$\text{Total Time} = T_{\text{init}} + T_{\text{comp}}$$

$$\text{Speedup } (S) = \frac{T_1}{T_p} \quad \text{where} \begin{cases} T_1 = \text{Time for 1 processor} \\ T_p = \text{Time for } P \text{ processors} \end{cases}$$

$$\text{Computation Speedup} = \frac{T_{\text{comp},1}}{T_{\text{comp},p}}$$

$$\text{Total Speedup} = \frac{T_{\text{total},1}}{T_{\text{total},p}}$$

3. A section in your report describing and analysing your solution. This should include at least:
   - A brief description of how your program works. Describe the general program flow and all significant data structures. How is the problem decomposed? How is the work assigned to threads? Where is the communication and synchronisation?
   - Execution time and speedup (both total and computation) for 1, 2, 4, ..N threads, where N is the max number of threads/processes of your machine. For most machines N might be 8
   - Discuss the results you expected and explain the reasons for any non-ideal behaviour you observe. In particular, if your program does not achieve perfect speedup, explain why. Is it due to work imbalance? Communication/synchronisation overhead? Performing redundant (or unnecessary) work? Is it possible to achieve a speedup better than perfect? Provide measurements to back up your explanations.

3.2 Task 2: Distributed Memory Parallel Programming Using MPI

Tip

You might want to refer to the practical tutorial, specifically the OpenMPI section, before starting.

Imagine that the company has grown, and the routing calculations have become too complex to run on the Nvidia Jetson Orin onboard the AETs. The engineering team has decided to offload computation to a distributed cloud cluster housed at the company's new data centre. The cluster has several nodes connected via Ethernet, each with its own private memory. You will replicate this environment on your PC, using MPI and the MMU to enforce the private memory constraints of the distributed nodes.

You are required to do the following:

1. Implement a parallel Branch-and-Bound program for the minimum-energy-consumption problem, aiming to achieve the best possible speed-up. Your program should accept the command-line arguments -p and -i to specify the number of processors used and the input file, respectively. It should also use the input file format described above. (File loading and command-line parsing are already implemented for you in the starter code).

2. Evaluate the performance of your program by measuring the following times using the provided gettime() functions that return the current time in seconds
   - Initialisation Time(T_{init}): The time required to read all the command-line arguments and create the separate processes.
   - Computation time(T_{comp}): Measures strictly the time to compute the answer, i.e you start timing when the main computation starts and end when the answer has been calculated.
3. A section in your report describing and analysing your solution. This should include at least:
   - A brief description of how your program works. Describe the general program flow and all significant data structures. How is the problem decomposed? How is the work assigned to threads? Where is the communication and synchronisation?
   - Execution time and speedup (both total and computation) for 1, 2, 4, ..N threads, where N is the max number of threads/processes of your machine. For most machines N might be 8
   - Discuss the results you expected and explain the reasons for any non-ideal behaviour you observe. In particular, if your program does not achieve perfect speedup, explain why. Is it due to work imbalance? Communication/synchronisation overhead? Performing redundant (or unnecessary) work? Is it possible to achieve a speedup better than perfect? Provide measurements to back up your explanations.
4. Compare the performance of your message-passing version of the minimum energy consumption problem to your shared-memory version of the application(Include graphs to illustrate the difference in performance.) Discuss any differences in performance with possible reasons for such behaviour.

Report Requirements

Format your report as a technical article. Include the following sections:
- **Introduction:** Briefly outline the practical aim, your implementation strategy (OpenMP and MPI), and the benchmarking goals.
- **Methodology:** Describe your Branch-and-Bound implementation approach for both OpenMP and MPI, the parallelisation strategy used in each, and the timing methodology.
- **Results:**Present tables and graphs comparing execution time and speedup across thread/process counts (1, 2, 4, . . . N) for both OpenMP and MPI.
- **Discussion:** : Analyse the performance differences between the OpenMP and MPI implementations. Discuss reasons for non-ideal speedup, communication or synchronisation overhead, and any other observed behaviour.
- **Conclusion:** Summarise findings, identify limitations, and suggest potential improvements for both implementations

4 Marking Rubric

Category	Component	Marks
Oral Exam (20%)	Quiz on your approach and prac fundamentals	20
Report (80%)	Introduction	10
	Methodology	15
	Results Presentation	15
	Discussion	25
	Conclusion	10
	Formatting	5
Total		**100**

Table 1: Mark Allocation for Practical 3

Appendix

This section gives a very brief introduction to the Branch and Bound algorithm. To explain its working we will make use of 4 client locations i.e Cape Town, Johannesburg, Pretoria and Durban.

The graph in Figure 1 below shows the energy costs between the 4 client locations:

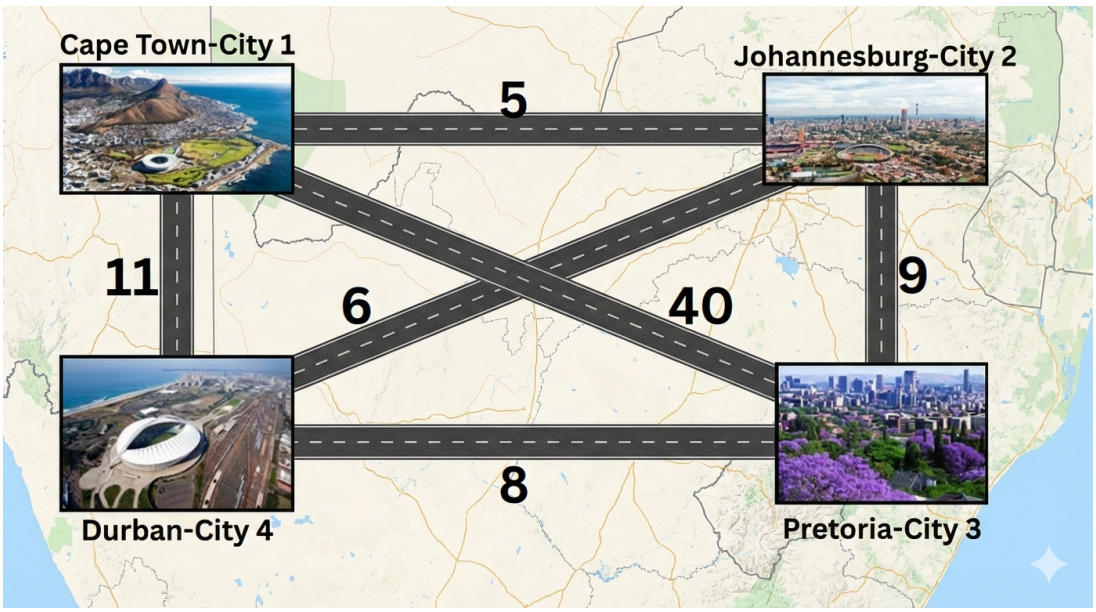

Figure 1: A graph of 4 client locations

Let's consider an exhaustive way of solving the minimum total energy consumption problem. One approach is to create a tree that describes all of the possible routes from City 1(Cape Town), as shown in the Figure 2 below:

For this 4 client example, E[1][2] = 5, E[1][3] = 40, E[1][4] = 11, E[2][3] = 9, E[2][4] = 6, E[3][4] = 8. All of the possible delivery routes can be found by travelling from the Central Distribution Hub to a different client location three times, once for each unique path and then back to the Central Distribution Hub. For example, by taking the middle branch from the Central Distribution

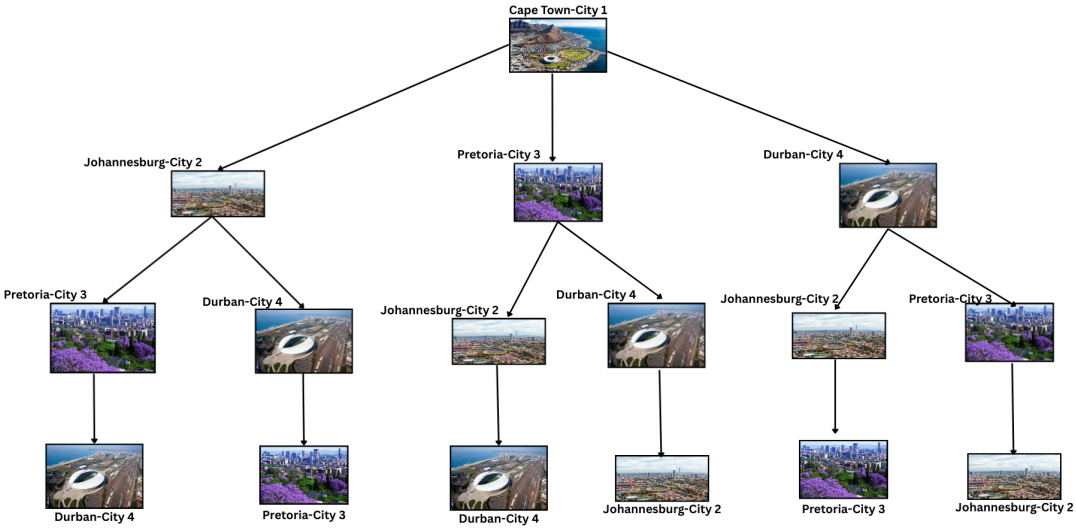

Figure 2: Branch and Bound Tree

Hub and the right branch after that, the delivery route $1 \rightarrow 3 \rightarrow 4 \rightarrow 2$ is produced, and has an energy cost of 40+8+6 = 54. There are six unique Central Distribution Hub to client location paths in this tree. Each possible delivery route is represented twice, once in each direction.

A simple solution would be to follow all the six delivery routes and determine the total energy cost for each and then select the delivery route with the most minimum energy cost.

A better solution would be to traverse the tree recursively as the summation of earlier parts of each delivery route is not repeated every time that delivery route portion is reused in several routes. The Branch and Bound approach uses this type of problem formulation but with added intelligence i.e it makes use of more knowledge of the problem to prune the tree as much as possible so that less evaluation is necessary. This can be summarised as follows:

1. Evaluate one route of the tree in its entirety (say $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$) and determine the energy cost of that path. Call this distance the current "bound" of the problem. The bound for this path in the above tree is 5 + 9 + 8 = 22.
2. Next, suppose that a second path is partially evaluated, say path $1 \rightarrow 3$, and the partial energy cost, 40, is already greater than the bound. If that is the case, then there is no need to complete the traversal of any part of the tree from there on, because all of those possible delivery routes (in this case there are two) must have an energy cost greater than the bound. In this way the tree is pruned and therefore does not have to be entirely traversed.
3. Whenever any route is discovered that has a better distance than the current bound, then the bound is updated to this new value.
4. The Branch-and-Bound approach always remembers the best path it has found so far, and uses that to prevent searching down parts of the tree that couldn't possibly produce better routes.